20

分页：更小的页表

现在我们来解决分页（paging）带来的第二个问题：页表（page table）太大，占用的内存太多。先从线性页表（linear page table）说起。你也许还记得[1]，线性页表会变得相当大。我们再假设一个 32 位（2^{32} 字节）的地址空间（address space），页大小（page size）为 4 KB（2^{12} 字节），每个页表项（page table entry，PTE）占 4 字节。这样，一个地址空间里大约有一百万（$2^{32} / 2^{12}$）个虚拟页（virtual page）；再乘以页表项的大小，就会发现页表足有 4 MB。还要记得：系统中通常每个进程（process）都有一张页表！如果有一百个活跃进程（这在现代系统上并不稀奇），光是页表就要占用几百 MB 内存！因此，我们需要一些技术来减轻这个沉重的负担。这类技术有很多，我们这就开始。不过，先来看看关键问题：

关键问题：如何让页表更小?
简单的基于数组的页表（通常称为线性页表）太大，在典型的系统上会占用太多内存。怎样才能让页表更小？关键思路是什么？这些新的数据结构（data structure）又会带来哪些低效之处?

20.1 简单的办法：更大的页

有一个简单的办法可以减小页表：用更大的页。还是以 32 位地址空间为例，但这次假设页大小为 16 KB。这样，虚拟页号（virtual page number，VPN）就是 18 位，偏移量（offset）是 14 位。假设每个 PTE 的大小不变（4 字节），线性页表现在有 2^{18} 个条目，每张页表总共 1 MB，缩小到原来的四分之一。不出所料，页表缩小的倍数正好等于页大小增大的倍数。

但这种方法的主要问题是，大页（large page）会导致每一页内部出现浪费，这个问题叫作**内部碎片**（internal fragmentation），因为浪费发生在分配单元的**内部**。结果，应用程序分配了一页又一页，却只用到每页的一小部分，内存很快就被这些过大的页占满。因此，大多数系统在通常情况下都使用比较小的页：4 KB（如 x86）或 8 KB（如 SPARCv9）。唉，我们的问题没法这么简单地解决。

[1]当然，你也可能不记得了；分页这东西是不是有点失控了？话虽如此，在着手解决之前，一定要先弄清楚你要解决的是什么问题；事实上，只要理解了问题，往往就能自己推导出解决办法。这里的问题应该很清楚：简单的线性（基于数组的）页表太大了。

补充：多种页大小

顺便提一下，现在许多体系结构（例如 MIPS、SPARC、x86-64）都支持多种页大小。通常用的是较小的页（page），4 KB 或 8 KB。不过，如果某个“聪明”的应用程序提出要求，也可以让地址空间中的某一部分使用一个大页（large page，例如 4 MB），这样应用程序就能把某个经常使用的（大型）数据结构放在这里，而只占用转换后备缓冲区（translation-lookaside buffer，TLB）中的一项。这种大页（large page）的用法在数据库管理系统（database management system）和其他高端商业应用中很常见。不过，支持多种页大小的主要原因并不是节省页表空间，而是减轻 TLB 的压力，让程序能访问更多的地址空间，又不至于产生太多 TLB 未命中（TLB miss）。但研究人员也指出 [N+02]，使用多种页大小会让操作系统的虚拟内存（virtual memory）管理器明显变得更复杂，因此有时最省事的做法，就是给应用程序提供一个新接口，让它们直接申请大页（large page）。

20.2 混合方案：分页和分段

生活中，只要某件事有两种合理但不同的做法，你就应该看看能不能把两者结合起来，做到两全其美。我们把这种结合称为**混合方案**（hybrid）。比如说，既然可以把巧克力和花生酱结合成一种美妙的混合体，也就是 Reese's 花生酱杯 [M28]，何必只吃巧克力或者只吃花生酱呢?

多年前，Multics 的设计者们（特别是 Jack Dennis）在构建 Multics 虚拟内存系统时，偶然想到了这样一个主意 [M07]。具体来说，Dennis 想到把分页和分段（segmentation）结合起来，以减少页表的内存开销。仔细看看一个典型的线性页表，就能明白为什么这样可行。假设某个地址空间里，堆（heap）和栈（stack）实际用到的部分都很小。我们用一个很小的 16 KB 地址空间来举例，页大小为 1 KB（图 20.1）；这个地址空间的页表如图 20.2 所示。

这个例子假设唯一的代码页（VPN 0）映射到物理页（physical page）10，唯一的堆页（VPN 4）映射到物理页 23，地址空间另一端的两个栈页（VPN 14 和 15）分别映射到物理页 28 和 4。从图中可以看出，页表的大部分都没有用上，全是**无效**（invalid）条目。真是浪费！而这还只是一个很小的 16 KB 地址空间。想象一下 32 位地址空间的页表，里面可能浪费掉多少空间！算了，还是别想了，太可怕了。

于是有了我们的混合方案：与其为进程的整个地址空间准备一张页表，为什么不给每个逻辑段（logical segment）各准备一张呢？在这个例子中，我们就可以有三张页表，分别对应地址空间中的代码、堆和栈部分。

回想一下，在分段中，我们有一个**基址**（base）寄存器，告诉我们每个段在物理内存（physical memory）中的位置，还有一个**界限**（bound）或**限制**（limit）寄存器，告诉我们这个段的大小。在混合方案中，内存管理单元（memory management unit，MMU）里仍然有这些结构；只不过这里的基址寄存器（base register）不再指向段本身，而是保存该段页表的物理地址（physical address）。界限寄存器（bounds register）则用来表示页表的末尾（即页表中有多少个有效页）。

 © 2008–23, ARPACI-DUSSEAU

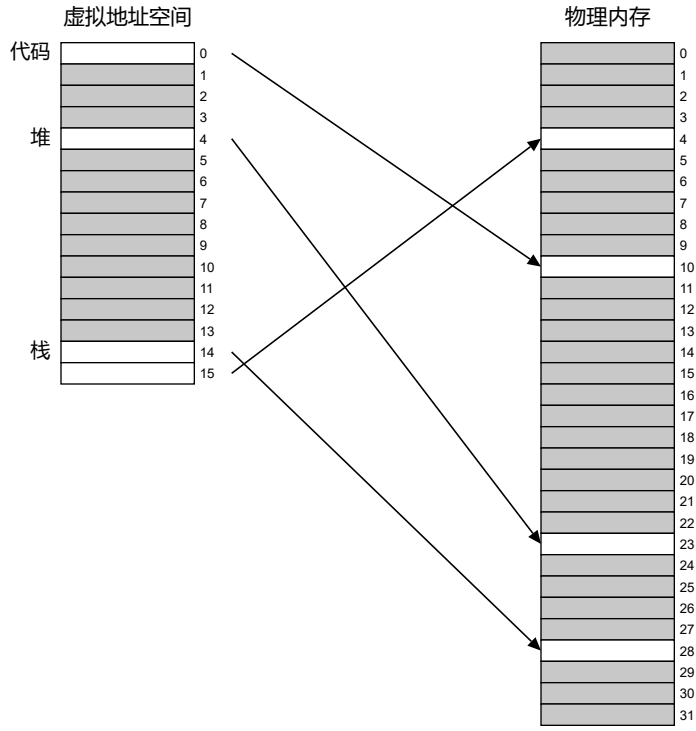


图 20.1：**页大小为 1 KB 的 16 KB 地址空间**

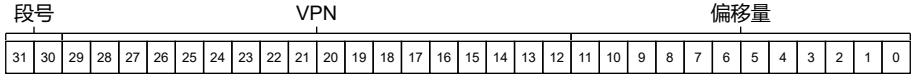
PFN	有效位	保护位	存在位	脏位
10	1	r-x	1	0
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
23	1	rw-	1	1
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
28	1	rw-	1	1
4	1	rw-	1	1

图 20.2：**16 KB 地址空间的页表**

 © 2008–23, ARPACI-DUSSEAU

我们用一个简单的例子来说明。假设有一个 32 位的虚拟地址空间（virtual address space），页大小为 4 KB，地址空间分成四个段。在这个例子中我们只用三个段：一个用于代码，一个用于堆，一个用于栈。

为了确定一个地址属于哪个段，我们用地址空间的最高两位来区分。假设 00 是未使用的段，01 表示代码，10 表示堆，11 表示栈。这样，虚拟地址（virtual address）的格式如下：



因此，假设硬件中有三对基址/界限寄存器（base/bounds register），代码、堆和栈各一对。进程运行时，每个段的基址寄存器都保存着该段线性页表的物理地址；这样，系统中的每个进程现在都关联着三张页表。发生上下文切换（context switch）时，必须修改这些寄存器，让它们指向新运行进程的页表所在的位置。

发生 TLB 未命中时（假设 TLB 由硬件管理，即由硬件负责处理 TLB 未命中），硬件用表示段号（segment number）的那几位（`SN`）来决定使用哪一对基址/界限寄存器（base/bounds register）。然后，硬件取出其中的物理地址，按如下方式和 VPN 组合，得到页表项（PTE）的地址：

```
SN            = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN           = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

这个过程应该很眼熟：它和我们之前看到的线性页表几乎一模一样。当然，唯一的区别是，这里用的是三个段基址寄存器中的一个，而不是单独的一个页表基址寄存器（page-table base register）。

我们这个混合方案的关键区别在于每个段都有一个界限寄存器，其中保存的是该段中最大有效页的值。例如，如果代码段（code segment）用了它的前三页（0、1 和 2），代码段的页表就只会分配三个条目，界限寄存器设为 3；访问超出段末尾的内存会产生异常，很可能导致进程被终止。这样一来，与线性页表相比，混合方案能节省大量内存：栈和堆之间未分配的页不再占用页表空间（以前占着空间，只是为了把它们标记为无效）。

不过你可能已经注意到，这种方法并非没有问题。第一，它仍然要求我们使用分段；正如之前讨论过的，分段并不像我们希望的那样灵活，因为它假定地址空间有某种特定的使用模式。比如，如果堆很大却用得很稀疏，页表还是会浪费很多空间。第二，这种混合方案会让外部碎片（external fragmentation）再次出现。虽然大部分内存是以页为单位管理的，但

提示：使用混合方案

当你有两个都不错、看起来又互相对立的想法时，总应该看看能不能把它们结合成一个**混合方案**（hybrid），做到两全其美。例如，杂交玉米品种公认比任何天然品种都更强健。当然，杂交并不总是好主意；看看斑驴（zeedonk，也叫 zonkey）就知道了，它是斑马和驴的杂交种。如果你不相信有这种动物，就去查一查，准备好大吃一惊吧。

页表的大小现在可以是任意的（PTE 大小的整数倍）。因此，要在内存中为页表找到空闲空间（free space）就更复杂了。出于这些原因，人们继续寻找更好的办法来实现更小的页表。

20.3 多级页表

另一种方法不依赖分段，但要解决的是同一个问题：怎样去掉页表中所有那些无效区域，而不是把它们全都留在内存里？我们把这种方法称为**多级页表**（multi-level page table），因为它把线性页表变成了类似树的结构。这种方法非常有效，许多现代系统都采用了它（例如 x86 [BOH10]）。下面我们详细介绍。

多级页表背后的基本思想很简单。首先，把页表按页大小切成若干块；然后，如果某一整页的页表项（PTE）都无效，就干脆不分配这一页页表。为了记录页表的每一页是否有效（如果有效，还要记录它在内存中的位置），我们使用一个新的结构，叫作**页目录**（page directory）。页目录既可以告诉你页表的某一页在哪里，也可以告诉你页表的某一整页里没有任何有效页。

图 20.3 给出了一个例子。图的左边是经典的线性页表；尽管地址空间中间的大部分区域都无效，我们仍然要为这些区域分配页表空间（即页表中间的两页）。右边是多级页表。页目录只把页表中的两页标记为有效（第一页和最后一页）；因此，只有页表的这两页驻留在内存中。由此，我们可以这样直观地理解多级页表：它只是让线性页表的一部分消失，腾出这些页帧（page frame）另作他用，并用页目录记录页表的哪些页已经分配。

在简单的两级页表中，页目录为页表的每一页提供一个条目。它由若干**页目录项**（page directory entries，**PDE**）组成。PDE（至少）包含一个**有效位**（valid bit）和一个**页帧号**（page frame number，PFN），这一点和 PTE 类似。但正如上面暗示的，这个有效位的含义稍有不同：如果 PDE 有效，就说明它（通过 PFN）指向的那一页页表中，至少有一个 PTE 的有效位被设为 1。如果 PDE 无效（即等于 0），PDE 的其余部分就是未定义的。

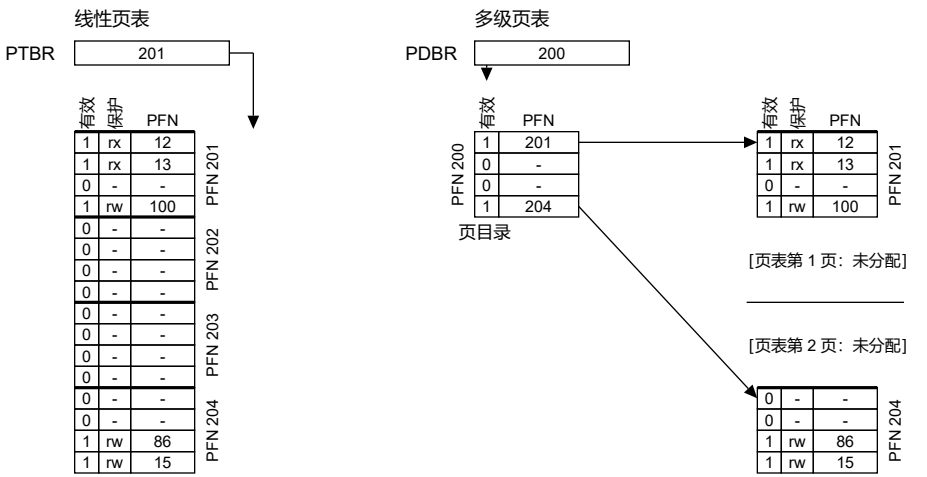


图 20.3：**线性页表（左）和多级页表（右）**

提示：理解时空权衡
构建数据结构时，总要考虑**时空权衡**（time-space trade-offs）。通常，如果你想让某个数据结构的访问更快，就得为它付出多占空间的代价。

与我们目前见过的方法相比，多级页表有一些明显的优点。第一，也许也是最明显的一点，多级页表分配的页表空间只与你实际使用的地址空间大小成正比；因此它通常很紧凑，并且支持稀疏地址空间。

第二，只要构造得当，页表的每一部分都正好能放进一页，这让内存管理更容易：操作系统在需要分配或增长页表时，只要拿下一个空闲页（free page）就行。相比之下，简单的线性页表本身不分页（non-paged）[2]，只是一个用 VPN 作索引的 PTE 数组；采用这种结构，整个线性页表必须连续地存放在物理内存中。对于比较大的页表（比如 4 MB）来说，要找到这么大一块连续的空闲物理内存可能相当困难。而在多级结构中，我们通过页目录增加了一个**间接层**（level of indirection），由页目录指向页表的各个部分；正是这层间接，让我们可以把页表的各页放在物理内存中任何想放的地方。

需要注意的是，多级页表是有代价的：发生 TLB 未命中时，要从内存加载两次才能从页表中拿到正确的转换信息（一次读页目录，一次读 PTE 本身），而线性页表只需加载一次。因此，多级页表是**时空权衡**（time-space trade-off）的一个小例子。我们想要更小的页表，也得到了，但这不是免费的：虽然在常见情况（common case），即 TLB 命中（TLB hit）时，性能显然完全一样，但页表变小之后，TLB 未命中的代价更高了。

另一个明显的缺点是复杂性。在 TLB 未命中时，无论由硬件还是操作系统来查找页表，查找过程都无疑比简单的线性页表更复杂。我们常常愿意增加复杂性来提高性能或降低开销；就多级页表而言，我们让页表查找变得更复杂，为的是节省宝贵的内存。

一个详细的多级页表示例

为了更好地理解多级页表背后的思想，我们来看一个例子。设想一个大小为 16 KB 的小地址空间，页大小为 64 字节。这样，我们就有一个 14 位的虚拟地址空间，其中 VPN 占 8 位，偏移量占 6 位。即使只用了地址空间的一小部分，线性页表也会有 2^8（256）个条目。图 20.4（第 7 页）展示了这样一个地址空间的例子。

在这个例子中，虚拟页 0 和 1 用于代码，虚拟页 4 和 5 用于堆，虚拟页 254 和 255 用于栈；地址空间的其余页都没有使用。

为了给这个地址空间构建两级页表，我们从完整的线性页表出发，把它按页大小切成若干块。回想一下，（在这个例子中）完整的页表有 256 个条目；假设每个 PTE 大小为 4 字节。因此，页表大小为 1 KB（256 × 4 字节）。由于页大小为 64 字节，1 KB 的页表可以分成 16 个 64 字节的页；每页可以容纳 16 个 PTE。

[2]这里我们做了一些假设：所有页表都完整地驻留在物理内存中（也就是说，它们没有被交换到磁盘上）；我们很快就会放宽这个假设。

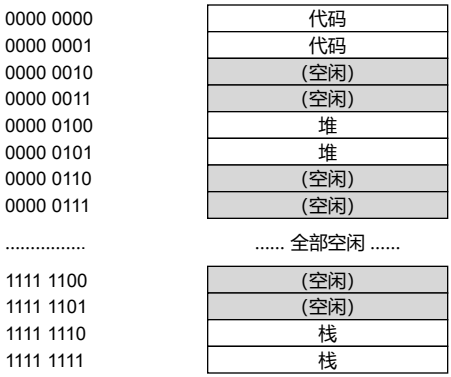
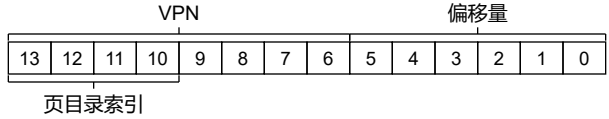


图 20.4：**页大小为 64 字节的 16 KB 地址空间**

现在我们需要弄清楚的是，拿到一个 VPN 后，怎样先用它索引页目录，再索引页表中的某一页。记住，两者都是条目数组；因此，我们只需弄清楚怎样用 VPN 的各部分分别构造出两者的索引。

我们先来索引页目录。这个例子中的页表很小：256 个条目，分布在 16 个页上。页目录需要为页表的每一页提供一个条目，因此它有 16 个条目。这样，我们就需要用 VPN 中的四位来索引页目录；我们使用 VPN 的最高四位，如下所示：



从 VPN 中取出**页目录索引**（page-directory index，简称 `PDIndex`）后，就可以通过一个简单的计算找到页目录项（PDE）的地址：`PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))`。这样就得到了页目录中的对应条目，接下来检查它，继续完成地址转换（address translation）。

如果页目录项（page directory entry）被标记为无效，我们就知道这次访问是无效的，于是引发异常。但如果 PDE 有效，就还有更多工作要做。具体来说，我们现在必须从这个页目录项（page directory entry）所指向的那一页页表中取出页表项（PTE）。为了找到这个 PTE，我们要用 VPN 的剩余位来索引这部分页表：



然后就可以用这个**页表索引**（page-table index，简称 `PTIndex`）来索引页表本身，得到 PTE 的地址：

```
PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
```

页目录		页表的一页（@PFN:100）			页表的一页（@PFN:101）		
PFN	有效?	PFN	有效位	保护位	PFN	有效位	保护位
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

图 20.5：**页目录以及页表的几个部分**

注意，从页目录项（page directory entry）中得到的页帧号（page frame number），必须先左移到正确的位置，再和页表索引（page-table index）组合，形成 PTE 的地址。

为了检验这一切是否讲得通，我们现在往多级页表中填入一些实际的值，然后转换一个虚拟地址。先从这个例子的**页目录**（page directory）看起（图 20.5 左边）。

从图中可以看到，每个页目录项（PDE）都描述了该地址空间页表中某一页的情况。在这个例子中，地址空间有两个有效区域（分别在开头和末尾），中间是许多无效映射。

物理页 100 是页表第 0 页所在的物理帧（physical frame），这一页共有 16 个页表项，对应地址空间中的前 16 个 VPN。这部分页表的内容见图 20.5（中间部分）。

页表的这一页包含前 16 个 VPN 的映射；在我们的例子中，代码段的 VPN 0 和 1 是有效的，堆的 4 和 5 也是。因此，表中有这几页各自的映射信息，其余条目都标记为无效。

页表的另一个有效页位于 PFN 101 中。这一页包含地址空间中最后 16 个 VPN 的映射；详见图 20.5（右边）。

在这个例子中，栈所在的 VPN 254 和 255 的映射是有效的。希望你能从这个例子中看出，多级索引（multi-level index）结构能节省多少空间。在这个例子中，我们不必为线性页表分配完整的十六页，只需分配三页：一页用于页目录，两页用于含有有效映射的页表块（page-table chunk）。对于大的（32 位或 64 位）地址空间，节省的空间显然会多得多。

最后，我们用这些信息来完成一次地址转换。这里有一个地址，它指向 VPN 254 的第 0 个字节：`0x3F80`，二进制为 `11 1111 1000 0000`。

回想一下，我们用 VPN 的最高 4 位来索引页目录。因此，`1111` 会选中上面页目录的最后一项（如果从第 0 项算起，就是第 15 项）。这一项指向位于地址 `101` 的一个有效的页表页。然后，我们用 VPN 的接下来 4 位（`1110`）索引这一页页表，找到所需的 PTE。1110 是这一页的倒数第二项（第 14 项），它告诉我们，虚拟地址空间的第 254 页映射到物理页 55。把 PFN=55（即十六进制 `0x37`）和 offset=000000 拼接起来，就得到了所需的物理地址，

提示：小心复杂性

系统设计者应当警惕给系统增加复杂性。优秀的系统构建者会用最简单的系统完成手头的任务。例如，如果磁盘空间很充裕，就不该设计一个费尽心思节省每一个字节的文件系统（file system）；同样，如果处理器很快，在操作系统中写一个清晰易懂的模块，比为手头的任务写出针对 CPU 极致优化的手工汇编代码要好。要警惕不必要的复杂性，无论它表现为过早优化（premature optimization）的代码还是别的形式；这类做法会让系统更难理解、维护和调试。安托万·德·圣埃克苏佩里（Antoine de Saint-Exupéry）有句名言："完美不是无以复加，而是无可删减。"他没写的是："谈论完美，要比真正做到完美容易得多。"

可以把请求发给内存系统了：`PhysAddr = (PTE.PFN << SHIFT) + offset = 00 1101 1100 0000 = 0x0DC0`。

现在你应该大致知道，如何用一个指向页表各页的页目录来构建两级页表了。不过遗憾的是，事情还没完。我们接下来会看到，有时两级页表还不够！

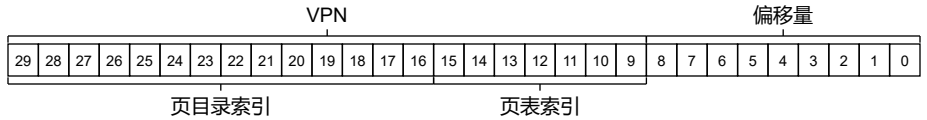
超过两级

到目前为止，我们的例子都假设多级页表只有两级：一个页目录，加上页表的若干部分。在某些情况下，树还可以更深（而且确实需要更深）。

我们用一个简单的例子来说明为什么更深的多级页表会有用。在这个例子中，假设有一个 30 位的虚拟地址空间，页很小（512 字节）。因此，虚拟地址中虚拟页号部分占 21 位，偏移量占 9 位。

记住我们构建多级页表的目标：让页表的每一部分都能放进一页。到目前为止，我们只考虑了页表本身；可是，如果页目录变得太大怎么办？

为了确定多级页表需要多少级才能让页表的所有部分都放进一页，我们首先要算出一页能放多少个页表项。页大小为 512 字节，假设 PTE 大小为 4 字节，你应该能算出一页能放 128 个 PTE。因此，索引页表中的某一页时，我们需要用 VPN 的最低 7 位（$\log_2 128$）作为索引：



从上图你可能还会注意到，留给（很大的）页目录的位有 14 位之多。如果页目录有 2^{14} 个条目（PDE 为 4 字节），它就不是只占一页，而是占 128 页，这样，让多级页表的每一部分都能放进一页的目标就落空了。

为了解决这个问题，我们给树再加一级：把页目录本身也拆分成多页，然后在它上面再加一个页目录，用来指向页目录的各页。于是，我们可以这样划分虚拟地址：



现在，索引上层页目录时，我们使用虚拟地址最高的那几位，即图中的“页目录索引 0”（page-directory index 0）；用这个索引可以从顶级页目录中取出页目录项（page directory entry）。如果它有效，就把顶级 PDE 中的物理帧号（physical frame number）和 VPN 的下一部分，即“页目录索引 1”（page-directory index 1）组合起来，查询第二级页目录。最后，如果这一项也有效，就把页表索引（page-table index）和第二级 PDE 中的地址组合起来，得到 PTE 的地址。呼！工作量真不小。而这一切，只是为了在多级页表中查找一个条目。

转换过程：别忘了 TLB

我们再一次以算法的形式给出控制流（图 20.6），总结一下用两级页表做地址转换的整个过程。这张图展示了*每一次*内存引用时硬件中发生的事情（假设 TLB 由硬件管理）。

从图中可以看出，在进行任何复杂的多级页表访问之前，硬件会先检查 TLB；如果命中，就像以前一样，直接构造出物理地址，完全不用访问页表。只有在 TLB 未命中时，硬件才需要执行完整的多级查找。在这条路径上，你可以看到传统两级页表的代价：要多访问两次内存，才能查到一个有效的转换。

20.4 反置页表

在页表的世界里，还有一种更极端的节省空间的办法：**反置页表**（inverted page tables）。这种方法不是为系统中每个进程各维护一张页表，而是只保留一张页表，系统中的每个*物理页*在其中都有一个条目。这个条目告诉我们，哪个进程在使用这一页，以及该进程的哪个虚拟页映射到这个物理页。

这样一来，要找到正确的条目，就得搜索这个数据结构。线性扫描的代价很高，因此通常会在这个基本结构之上再建一张散列表（hash table），以加快查找。PowerPC 就是这类体系结构的一个例子 [JM98]。

更一般地说，反置页表（inverted page table）说明了我们从一开始就在强调的一点：页表只不过是数据结构。你可以对数据结构做很多疯狂的事情：让它们更小或更大，更慢或更快。多级页表和反置页表（inverted page table）只是众多可能做法中的两个例子。

20.5 把页表交换到磁盘

最后，我们再放宽一个假设。到目前为止，我们一直假设页表驻留在内核拥有的物理内存中。然而，即使用了这么多技巧来减小页表，页表仍然可能大到无法一次全部放进内存。因此，有些系统把这些页表放在**内核虚拟内存**（kernel virtual memory）中，这样在内存有点紧张时，系统就可以把其中一部分页表**交换**（swap）到磁盘上。等我们更详细地了解了如何把页移进移出内存之后，会在后面的章节（即 VAX/VMS 的案例研究）中进一步讨论这个问题。

 © 2008–23, ARPACI-DUSSEAU

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)    // TLB 命中
4   if (CanAccess(TlbEntry.ProtectBits) == True)
5     Offset   = VirtualAddress & OFFSET_MASK
6     PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7     Register = AccessMemory(PhysAddr)
8   else
9     RaiseException(PROTECTION_FAULT)
10 else                    // TLB 未命中
11   // 先取页目录项
12   PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13   PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14   PDE     = AccessMemory(PDEAddr)
15   if (PDE.Valid == False)
16     RaiseException(SEGMENTATION_FAULT)
17   else
18     // PDE 有效：再从页表中取 PTE
19     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
20     PTEAddr = (PDE.PFN<<SHIFT) + (PTIndex*sizeof(PTE))
21     PTE     = AccessMemory(PTEAddr)
22     if (PTE.Valid == False)
23       RaiseException(SEGMENTATION_FAULT)
24     else if (CanAccess(PTE.ProtectBits) == False)
25       RaiseException(PROTECTION_FAULT)
26     else
27       TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
28       RetryInstruction()
```

图 20.6：**多级页表的控制流**

20.6 小结

我们已经看到了真实的页表是如何构建的：不一定只是线性数组，也可以是更复杂的数据结构。这类页表需要在时间和空间之间权衡：页表越大，处理 TLB 未命中就越快，反之亦然。因此，选择什么样的结构，在很大程度上取决于具体环境的约束。

在内存受限的系统中（比如许多较老的系统），小的结构比较合理；而在内存比较充足、工作负载（workload）又会频繁使用大量页的系统中，较大的页表能加快 TLB 未命中的处理，可能才是正确的选择。有了软件管理的 TLB（software-managed TLB），数据结构的整个设计空间都向操作系统创新者（提示：就是你）敞开了，任你施展。你能想出哪些新的结构？它们能解决什么问题？入睡时想想这些问题，做只有操作系统开发者才会做的大梦吧。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BOH10] "Computer Systems: A Programmer's Perspective" by Randal E. Bryant and David R. O'Hallaron. Addison-Wesley, 2010. *We have yet to find a good first reference to the multi-level page table. However, this great textbook by Bryant and O'Hallaron dives into the details of x86, which at least is an early system that used such structures. It's also just a great book to have.*

[JM98] "Virtual Memory: Issues of Implementation" by Bruce Jacob, Trevor Mudge. IEEE Computer, June 1998. *An excellent survey of a number of different systems and their approach to virtualizing memory. Plenty of details on x86, PowerPC, MIPS, and other architectures.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. *A terrific paper about a real virtual memory manager in a classic operating system, VMS. So terrific, in fact, that we'll use it to review everything we've learned about virtual memory thus far a few chapters from now.*

[M28] "Reese's Peanut Butter Cups" by Mars Candy Corporation. Published at stores near you. *Apparently these fine confections were invented in 1928 by Harry Burnett Reese, a former dairy farmer and shipping foreman for one Milton S. Hershey. At least, that is what it says on Wikipedia. If true, Hershey and Reese probably hate each other's guts, as any two chocolate barons should.*

[N+02] "Practical, Transparent Operating System Support for Superpages" by Juan Navarro, Sitaram Iyer, Peter Druschel, Alan Cox. OSDI '02, Boston, Massachusetts, October 2002. *A nice paper showing all the details you have to get right to incorporate large pages, or **superpages**, into a modern OS. Not as easy as you might think, alas.*

[M07] "Multics: History" Available: http://www.multicians.org/history.html. *This amazing web site provides a huge amount of history on the Multics system, certainly one of the most influential systems in OS history. The quote from therein: "Jack Dennis of MIT contributed influential architectural ideas to the beginning of Multics, especially the idea of combining paging and segmentation." (from Section 1.2.1)*

作业（模拟）

这个有趣的小作业用来检验你是否理解多级页表的工作原理。没错，上一句用“有趣”这个词是否恰当，还有些争议。这个程序叫 `paging-multilevel-translate.py`，名字大概不出你所料；详见 README。

问题

1. 使用线性页表时，假设 TLB 未命中时由硬件查找页表，你只需要一个寄存器来定位页表。要定位两级页表，需要几个寄存器？三级页表呢？
2. 用模拟器分别以随机种子 0、1 和 2 做地址转换，并用 `-c` 选项检查你的答案。每次查找需要多少次内存引用？
3. 根据你对缓存（cache）工作原理的理解，你认为对页表的内存引用在缓存中会表现如何？它们会带来大量缓存命中（cache hit），从而访问很快吗？还是会带来大量未命中，从而访问很慢？

 © 2008–23, ARPACI-DUSSEAU